PEA – projekt nr 2

Temat: <u>Implementacja i analiza efektywności algorytmu podziału i ograniczeń</u>

Należy zaimplementować oraz dokonać analizy efektywności algorytmu podziału i ograniczeń (B&B) dla asymetrycznego problemu komiwojażera (ATSP).

<u>Podczas realizacji zadania należy przyjąć następujące założenia:</u>

- używane struktury danych powinny być alokowane dynamicznie (w zależności od aktualnego rozmiaru problemu),
- program powinien umożliwić weryfikację poprawności działania algorytmu. W tym celu powinna istnieć możliwość wczytania danych wejściowych z pliku tekstowego,
- po zaimplementowaniu i sprawdzeniu poprawności działania algorytmu należy dokonać pomiaru czasu jego działania w zależności od rozmiaru problemu N (badania należy wykonać dla minimum 10 różnych reprezentatywnych wartości N),
- dla każdej wartości N należy wygenerować po 100 losowych instancji problemu (w sprawozdaniu należy umieścić tylko wyniki uśrednione – pamiętać, aby nie mierzyć czasu generowania instancji),
- należy przyjąć dopuszczalny czas wykonania dla B&B – np. pięć minut i jeśli problem nie został rozwiązany w tym czasie, to go przerwać - policzyć ile (w %) w zależności od N(rozmiar problemu) zostało przerwanych – dane umieścić w tabeli
- oszacować maksymalne średnie N przy założonym czasie np. 5 minut
- należy zaimplementować wybrane metody przeglądania przestrzeni rozwiązań (zależne od wariantu): *breadth-first-search*, *depth-first-search* i *lowest-cost* (*best-first-search*)
- zbadać wpływ rozwiązania początkowego na czas wykonania algorytmu
- implementacje algorytmów należy dokonać zgodnie z obiektowym paradygmatem programowania,
- używanie „okienek” nie jest konieczne i nie wpływa na ocenę (wystarczy wersja konsolowa),
- kod źródłowy powinien być komentowany.

<u>Sprawozdanie powinno zawierać:</u>

- wstęp teoretyczny zawierający opis rozpatrywanego problemu, oraz oszacowanie jego złożoności obliczeniowej na podstawie literatury dla badanych algorytmów,
- przykład praktyczny - opis działania algorytmu „krok po kroku” dla przykładowej instancji danego problemu o małej wartości N – minimum N=4 (proszę dobrać własny przykład, w którym jeden z elementów macierzy będzie miał wartość zdefiniowaną przez dwie ostatnie cyfry nr indeksu.

Odległości dróg powinny być w zakresie 10..99. Wybrać jedną z implementowanych metod przeszukiwania.

- opis implementacji algorytmu - dokładny opis funkcji obliczającej ograniczenie, opis istotnych aspektów implementacji
- wyniki eksperymentów (w postaci tabel i wykresów) – należy też porównać wyniki dla różnych metod (najlepiej na wspólnym wykresie)
- wnioski dotyczące otrzymanych wyników (odnieść się także do czasów Brute Force z poprzedniego zadania),
- kod źródłowy w formie elektronicznej wraz z wersją wykonywalną programu (kopiowany na dysk google'a).

<u>Sprawdzenie poprawności zaimplementowanego algorytmu:</u>

Aby sprawdzić poprawność działania algorytmu musi być możliwość wczytania danych z pliku tekstowego i wykonania na nich obliczeń. Menu programu powinno umożliwiać

1.Wczytanie danych z pliku

2.Wygenerowanie danych losowych

3.Wyświetlenie ostatnio wczytanych lub wygenerowanych danych

4.Uruchomienie danego algorytmu dla ostatnio wczytanych lub wygenerowanych danych i wyświetlenie wyników (należy wyświetlić długość ścieżki, ciąg wierzchołków oraz **czas wykonania algorytmu**)

Dane dla których będzie testowana poprawność algorytmu umieszczone są na stronie prowadzącego i/lub na dysku google.

Format danych w pliku jest następujący (analogicznie jak dla zadania 1):

- w pierwsze linii jest podana ilość miast,
- w pozostałych liniach macierz kosztów: w każdej linii wiersz macierzy (liczby przedzielone spacją),
- dane na przekątnej mają wartość równą -1
- wartość -1 na innej pozycji oznacza brak połączenia

Ocena (maksimum): zaczynamy od oceny 2.0 i dodajemy do tego wyniku punkty jak poniżej:

a) 3,0 - jedna metoda przeszukiwania przestrzeni (*depth-first-search*)
b) 3,5 – dwie metody przeszukiwania przestrzeni stanów (*best-first-search* i *breadth-first-search*)
c) 4,0 – dwie metody ale bez bibliotek (*best-first-search* i *breadth-first-search*)
d) 4,5 – trzy metody, ale bez bibliotek

e) 5,0 – trzy metody bez bibliotek, ale metoda powinna *best-first-search* być zrobiona algorytmem Little'a

Języki programowania

Wskazane jest korzystanie z języków kompilujących się do kodu maszynowego (np. C++).

Uwaga!!!

1.Jeżeli korzystamy z Visual Studio (lub innego środowiska), to testy prowadzimy na wersji RELEASE (a nie DEBUG)

2.Należy wgrać na dysk google'a **przed** oddaniem wersję exe programu, gdyż w tym zadaniu algorytmy innych osób będą testowane w tych samych warunkach – ich czas działania będzie wpływał na ocenę końcową.

Dodatkowe materiały internetowe:

https://www.ii.uni.wroc.pl/~prz/2011lato/ah/opracowania/met_podz_ogr.opr.pdf- plecakowy i TSP

http://cs.pwr.edu.pl/zielinski/lectures/om/mow10.pdf - TSP(Little'a) oraz B&B

https://www.youtube.com/watch?v=-cLsEHP0qt0 – wprowadzenie do TSP

https://www.youtube.com/watch?v=nN4K8xA8ShM&t=927s – TSP B&B

https://dspace.mit.edu/bitstream/handle/1721.1/46828/algorithmfortrav00litt.pdf?sequence=1&isAllowed=y - algorytm Little'a

plik ***komiw_little.pdf*** w materiałach na dysku google'a (tam, gdzie opis zadań)